

SMART VIRTUAL PIE & MICRO-GRID BOT (PRYWATNY ETF)

Ostateczna Specyfikacja Techniczna (PRD) – Wersja 2.3

SPIS TREŚCI

- 1. Architektura Systemu i Schemat Bazy Danych
- 2. Moduł A – Smart Virtual Pie
- 3. Moduł B – Micro-Grid Bot
- 4. Integracja API oraz Reconciliation
- 5. Embedded AI Chat Console
- 6. Porównanie z Botami na Binance
- 7. Analiza Opłat Transakcyjnych

1. ARCHITEKTURA SYSTEMU I SCHEMAT BAZY DANYCH

System działa w kontenerze Docker. Baza danych to lokalne SQLite. Komunikacja i powiadomienia realizowane są przez agenta OpenClaw (Moltbot) połączonego z lokalnym Ollama oraz Telegramem.

1.1 Struktura tabel SQL (gotowa do wdrożenia)

```
-- 1. Wirtualne koszyki (Prywatne ETF-y)
CREATE TABLE pies (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  name TEXT NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 2. Aktywa w koszykach + kontrola bota
CREATE TABLE pie_assets (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  user_id INTEGER NOT NULL,
  pie_id INTEGER NOT NULL,
  ticker TEXT NOT NULL,
  display_ticker TEXT NOT NULL,
  currency TEXT NOT NULL,
  target_weight REAL NOT NULL,
  is_bot_allowed INTEGER DEFAULT 0,  -- 0 = tylko manual, 1 = bot aktywny
  is_penny_stock INTEGER DEFAULT 0,  -- 1 = twarda blokada systemowa
  FOREIGN KEY (pie_id) REFERENCES pies(id) ON DELETE CASCADE
);

-- 3. Transakcje bota (zgrupowane w serie DCA)
CREATE TABLE active_trades (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  user_id INTEGER NOT NULL,
  position_group_id TEXT NOT NULL,  -- UUID v4
  ticker TEXT NOT NULL,
  currency TEXT NOT NULL,
  buy_order_id TEXT NOT NULL,
  sell_order_id TEXT,
  buy_price REAL NOT NULL,
  quantity REAL NOT NULL,
  allocated_value REAL NOT NULL,
  average_price REAL NOT NULL,
  dca_level INTEGER DEFAULT 0,
  status TEXT DEFAULT 'OPEN',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  closed_at TIMESTAMP
);

-- 4. Konfiguracja ryzyka + tryb Paper Trading
CREATE TABLE risk_settings (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```

    user_id INTEGER NOT NULL,
    dca_scenario TEXT DEFAULT '1,1,1,1,1',
    max_dca_levels INTEGER DEFAULT 5,
    max_spread_pct REAL DEFAULT 0.05,
    take_profit_usd REAL DEFAULT 0.05,
    max_daily_loss REAL DEFAULT 10.0,
    is_paper_trading INTEGER DEFAULT 1, -- 1 = Paper Trading (domyślnie)
    is_bot_active INTEGER DEFAULT 0
);

-- 5. Pełny Audit Log (zintegrowany)
CREATE TABLE audit_log (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER NOT NULL,
    position_group_id TEXT,
    action_type TEXT NOT NULL CHECK(action_type IN ('BUY', 'CANCEL', 'RELOAD', 'ERROR', 'INFO')),
    message TEXT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Indeksy
CREATE INDEX idx_pie_assets_pie_id ON pie_assets(pie_id);
CREATE INDEX idx_active_trades_user ON active_trades(user_id);
CREATE INDEX idx_active_trades_group ON active_trades(position_group_id);
CREATE INDEX idx_active_trades_status ON active_trades(status, ticker);

```

Alternatywna wersja SQL (propozycja od innego modelu):

```

-- 1. Główna tabela wirtualnych koszyków (Prywatnych ETF-ów)
CREATE TABLE pies (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 2. Spółki przypisane do koszyków + kontrola bota
CREATE TABLE pie_assets (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    pie_id INTEGER NOT NULL,
    ticker TEXT NOT NULL,
    display_ticker TEXT NOT NULL,
    currency TEXT NOT NULL,
    target_weight REAL NOT NULL,
    is_bot_allowed INTEGER DEFAULT 0,
    is_penny_stock INTEGER DEFAULT 0,
    FOREIGN KEY (pie_id) REFERENCES pies(id) ON DELETE CASCADE
);

-- 3. Transakcje bota skalpującego (zgrupowane w serie DCA)
CREATE TABLE active_trades (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    position_group_id TEXT NOT NULL,
    ticker TEXT NOT NULL,
    currency TEXT NOT NULL,
    buy_order_id TEXT NOT NULL,
    sell_order_id TEXT,
    buy_price REAL NOT NULL,
    quantity REAL NOT NULL,
    allocated_value REAL NOT NULL,
    average_price REAL NOT NULL,
    dca_level INTEGER DEFAULT 0,
    status TEXT DEFAULT 'OPEN',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    closed_at TIMESTAMP
);

-- 4. Twarde limity ryzyka i globalna konfiguracja bota
CREATE TABLE risk_settings (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    dca_scenario TEXT DEFAULT '1,1,1,1,1',
    max_dca_levels INTEGER DEFAULT 5,
    max_spread_pct REAL DEFAULT 0.05,

```

```

take_profit_usd REAL DEFAULT 0.05,
max_daily_loss REAL DEFAULT 10.0,
is_paper_trading INTEGER DEFAULT 1,
is_bot_active INTEGER DEFAULT 0
);

-- 5. Tabela rejestrująca wszystkie decyzje bota (Audit Log)
CREATE TABLE audit_log (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  position_group_id TEXT,
  action_type TEXT NOT NULL CHECK(action_type IN ('BUY', 'CANCEL', 'RELOAD', 'ERROR', 'INFO')),
  message TEXT NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Indeksy
CREATE INDEX idx_pie_assets_pie_id ON pie_assets(pie_id);
CREATE INDEX idx_active_trades_group ON active_trades(position_group_id);
CREATE INDEX idx_active_trades_status ON active_trades(status, ticker);

```

2. MODUŁ A – SMART VIRTUAL PIE

Umożliwia wygodne dokupywanie akcji „luzem” bez opłaty FX 0.15%.

- Elastyczne wagi – działają jako proporcje (nie sztywne %).
- Real-time 3-Way Sync – suwak, kwota i ilość przeliczają się wzajemnie. Obok każdego aktywa znajduje się ikonka Bota.
- Mini-wykresy – dane z yfinance, bez obciążania API T212.
- Smart FX Guard – ostrzeżenie, gdy brakuje środków w danej walucie.

3. MODUŁ B – MICRO-GRID BOT

Automatyczna pętla skalpująca mikro-kwotami (np. 1 USD) z precyzyjną obsługą awarii.

3.1 Strategia wejścia (dca_level = 0)

- Spread Guard: sprawdzenie spreadu przed wejściem.
- Źródło ceny: Domyślnie Finnhub (real-time). Yahoo Finance jako fallback (z opóźnieniem).
- Natychmiastowe wystawienie LIMIT SELL. Target Price = (allocated_value + take_profit_usd) / quantity (docelowy zysk z całej pozycji, nie dodatek do ceny akcji).

3.2 Strategia DCA – Pętla Cancel-Replace z Fail-Safe

Przy spadku ceny bot wykonuje sztywną sekwencję:

Krok 1 – CANCEL

Anulowanie aktualnego zlecenia LIMIT SELL. Jeśli wystąpi błąd/timeout – proces zostaje przerwany, błąd logowany jest do audit_log, wysyłany jest alert na Telegram. Zakup nie rusza bez potwierdzenia anulowania.

Krok 2 – BUY (z Fail-Safe)

Zakup kolejnego kroku DCA. W przypadku błędu API lub timeoutu bot natychmiast sprawdza cenę rynkową:

- Scenariusz A (zysk): Jeśli cena wzrosła i pozycja jest na plusie – bot rezygnuje z uśredniania i wykonuje MARKET SELL, inkasując zysk.
- Scenariusz B (brak zysku): Bot przywraca stare zlecenie LIMIT SELL na dotychczasową ilość i cenę, chroniąc pozycję.

Kroki 3-4 – RECALCULATE + PLACE nowego LIMIT SELL na całą skumulowaną pozycję.

3.3 Ochrona przed nadmiernym uśrednianiem

Maksymalna liczba poziomów DCA jest sztywno ograniczona parametrem max_dca_levels. Po osiągnięciu limitu bot przechodzi w tryb HODL.

3.4 Funkcja RELOAD (Manual Boost)

Użytkownik może ręcznie zresetować licznik dca_level do 0 dla zablokowanej serii. Nowy zakup zachowuje ten sam position_group_id.

3.5 Ikonka Bota i Penny Stock Guard

- Ikonka Bota w UI steruje flagą `is_bot_allowed` (decyzja manualna per ticker).
- System automatycznie blokuje aktywację bota dla spółek o niskiej płynności (`is_penny_stock = 1`).
- Dodatkowo przed aktywacją bota na tickerze sprawdzane jest programistycznie, czy dany instrument wspiera fractional shares (Fractional Guard).

4. INTEGRACJA API ORAZ RECONCILIATION

- Zlecenia wysyłane jako pojedyncze Value Orders.
- Reconciliation Loop (przy starcie): Bot porównuje stan bazy z rzeczywistymi zleceniami na koncie T212. Jeśli zlecenie `sell_order_id` zostało wykonane podczas niedostępności bota – seria jest automatycznie rozliczana jako CLOSED.

Lokalne endpointy dla OpenClaw:

- GET `/api/bot/status`
- POST `/api/bot/toggle`
- POST `/api/pie/reinvest`
- POST `/api/bot/reload`

5. EMBEDDED AI CHAT CONSOLE

- Bezpośrednie połączenie z lokalnym Ollama (pełna prywatność).
- Function Calling – sterowanie botem naturalnym językiem (włączanie, RELOAD, sprawdzanie pozycji).
- Live Web Search z automatycznym cytowaniem źródeł.

6. ZARZĄDZANIE KLUCZAMI API I USTAWIENIA ADMINA

Ponieważ system działa w modelu multi-user, wprowadzono mechanizmy zarządzania kluczami API oraz limitami na poziomie administratora.

6.1 Klucze API per użytkownik + fallback

- Każdy użytkownik może podać własne klucze API (np. Finnhub) w ustawieniach konta.
- Jeśli użytkownik nie poda własnego klucza — system korzysta z ****globalnego klucza**** administratora.
- Bot zawsze najpierw sprawdza klucz użytkownika, a dopiero potem klucz globalny.

6.2 Ustawienia administratora

Administrator ma dostęp do panelu, w którym może:

- Ustawiać maksymalną liczbę aktywnych botów na użytkownika.
- Zarządzać globalnymi kluczami API (Finnhub, inne źródła danych).
- Przeglądać statystyki użycia kluczy API.
- Wymuszać korzystanie wyłącznie z kluczy globalnych (dla wybranych użytkowników).

7. PORÓWNANIE Z BOTAMI NA BINANCE

Nasz bot oferuje znacznie większą kontrolę, bezpieczeństwo i elastyczność niż gotowe rozwiązania Binance.

Kryterium	Nasz Bot	Binance
Kontrola nad logiką	Pełna (własny Cancel-Replace + Fail-Safe)	Ograniczona
Bezpieczeństwo	Wysokie (Paper Trading, Reconciliation, Audit Log)	Średnie
Prywatność	Wysoka (lokalny Ollama)	Niska

Obsługa awarii DCA	Inteligentna (zysk → MARKET SELL / strata → przywraca SELL)	Podstawowa
Virtual Pie	Zaawansowany + ikonka bota per aktywo	Podstawowy Rebalancing

Podsumowanie: Nasz bot jest rozwiązaniem dla osób, które chcesz mieć maksymalną kontrolę, zaawansowane mechanizmy bezpieczeństwa i możliwość budowania własnych strategii. Boty Binance są prostsze, ale znacznie bardziej ograniczone.

8. ANALIZA OPŁAT TRANSAKCYJNYCH

Przy handlu wyłącznie w jednej walucie bazowej (USD lub EUR) na Trading 212 prowizja wynosi 0%, a FX Fee również wynosi 0%. Głównym kosztem jest spread.

Element	Trading 212	Binance (z BNB)
Prowizja	0%	0.075%
FX Fee (jedna waluta)	0%	0%
Główny koszt	Spread	Prowizja + Spread

Wniosek: Przy naszym stylu handlu (mikro-transakcje w jednej walucie) Trading 212 jest obecnie tańszy. Głównym ukrytym kosztem jest spread, na którym Trading 212 zarabia.

9. ZAŁĄCZNIK: ULEPSZONY PROMPT DO CLAUDE (WERSJA 2)

Poniżej znajduje się ulepszona wersja promptu do Claude'a, uwzględniająca wszystkie poprawki architektoniczne, multi-user oraz poprawiony wzór na Target Price.

Cześć Claude!

Dziękujemy za poprzednią, bardzo trafną recenzję. Masz rację — pierwotna koncepcja opierała się na nieistniejącym endpointzie Value Orders

KLUCZOWE DECYZJE ARCHITEKTONICZNE (Multi-User)

Nasz system jest w pełni multi-user. Każdy użytkownik:

- Ma własne wirtualne koszyki (pies)
- Ma własne pozycje i serie DCA (active_trades)
- Ma własne ustawienia ryzyka (risk_settings)
- Ma dedykowany telegram_chat_id do powiadomień
- Może posiadać własny klucz API Trading 212 (zaszyfrowany w bazie)

Bot działa jako wspólny proces w tle, który iteruje po aktywnych użytkownikach (is_bot_active = 1) i wykonuje logikę dla każdego z nich osobno

1. REZYGNACJA Z VALUE ORDERS

Trading 212 nie udostępnia w publicznym API możliwości składania zleceń po wartości. Rozwiązanie: Bot przed zakupem pobiera aktualną cenę

2. TARGET PRICE — POPRAWIONY WZÓR

Parametr take_profit_usd oznacza docelowy zysk nominalny z całej serii, a nie dodatek do ceny akcji.

Wzór na cenę zlecenia LIMIT SELL:

$\text{Target_Price} = (\text{SUM}(\text{allocated_value}) + \text{take_profit_usd}) / \text{SUM}(\text{quantity})$

3. CONTROLLED DCA + WEEKLY NAG GUARD

Zamiast automatycznego zamykania pozycji, wprowadzamy system powiadomień. Jeśli pozycja ma status OPEN dłużej niż 7 dni, bot raz w tydzień

4. FRACTIONAL GUARD

Przed aktywacją bota na danym tickerze (is_bot_allowed = 1) system sprawdza programistycznie, czy ticker wspiera fractional shares na Trading 212

5. ARCHITEKTURA

Rozbudowujemy istniejący projekt Sniper. Tworzymy dwa nowe moduły: `module_virtual_pie/` oraz `module_micro_grid/`. SQLite uruchamiamy w

6. SCHEMAT BAZY DANYCH (MULTI-USER)

[Pełny SQL z `user_id` we wszystkich tabelach + tabela `user_api_keys` — patrz PDF]

7. LOGIKA BOTA W TLE

Bot iteruje po użytkownikach z `is_bot_active = 1`, odszyfrowuje ich klucz API i wykonuje logikę Micro-Grid dla ich pozycji.